

Assignment #3

B+ Tree Implement

2019060546

주원웅

• Design

1. 가장 우선적으로 헤더파일의 구조체 분석을 하였다.

레코드(record) 구조체의 경우 **LEAF Node** 에서 저장할 수 있는 단위로 한 튜플을 저장할 수 있는 구조체이다.

```
typedef struct record
{
    int64_t key;
    char value[120];
} record;
```

인터레코드(I_R)는 B+Tree 의 **Non LEAF Node** 에서 저장할 수 있는 단위로, key 와 offset을 저장하는 구조체이다.

```
typedef struct inter_record
{
    int64_t key;
    off_t p_offset;
} I_R;
```

Page 구조체는 한 **Node** 를 표현하며, **LEAF, Non LEAF** 둘로 구분된다. 둘을 구분하는 멤버변수 `is_leaf` 가 존재하며, **leaf** 노드의 경우 저장할 수 있는 변수가 **record**의 **배열**, **nonleaf** 노드의 경우 저장할 수 있는 변수가 **I_R**의 **배열** 두가지로 나뉜다. **Next_offset** 멤버 변수의 경우 **nonleaf** 에서는 가장 앞의 자식 노드를 가리키는 포인터로 사용하였고, **leaf**의 경우 다음 **leaf** 노드를 가리키는 포인터로 사용하였다.

```
typedef struct Page
{
    off_t parent_page_offset;
    int is_leaf;
    int num_of_keys;
    char reserved[104];
    off_t next_offset;
    union
    {
        I_R b_f[248];           // inner -> panout
        record records[31];    // leaf_node -> real record
    };
} page;
```

2. 과제 명세의 요구사항에 맞게 **key-rotation** 을 구현하기 위해서 현재 **Leaf Node** 에서 왼쪽 오른쪽 **Sibling Leaf Node** 를 모두 확인하도록 구현할 계획이다. 또한 가장 중요한 **Root node** 에서의 처리를 따로 해주어야 했다. Delete, insert 작업시 key 를 찾는 알고리즘은 이진탐색을 사용하려 하였지만, **Split, rotation, Merge, Distribution** 등 leaf 의 **key** 를 옮겨야 하는 작업이 발생할 수 있으므로, 옮기는 작업과 찾는 작업이 두 번 발생하지 않도록, 옮기면서 찾는 작업을 할 수 있는 선형탐색을 사용하였다.

• Implement

1. db_Find 함수

```
char *db_find(int64_t key)
{
    if (rt == NULL)
        return 0;

    page *cur = load_page(rp->rpo);
    off_t cur_off = rp->rpo; // 없어도 됨
    off_t prev_off = 0;

    char *result = (char *)malloc(sizeof(char) * 120);
    while (!cur->is_leaf)
    {
        if (cur->parent_page_offset != prev_off)
        {
            cur->parent_page_offset = prev_off;
            pwrite(fd, cur, sizeof(page), cur_off);
        }
        if (key < cur->b_f[0].key)
        {
            prev_off = cur_off;
            cur_off = cur->next_offset;
            cur = load_page(cur->next_offset);
        }
        else
        {
            int flag = 0;
            for (int i = 1; i < cur->num_of_keys; i++)
            {
                if (key < cur->b_f[i].key)
                {
                    prev_off = cur_off;
                    cur_off = cur->b_f[i - 1].p_offset;
                    cur = load_page(cur->b_f[i - 1].p_offset);
                    flag = 1;
                    break;
                }
            }
            if (!flag)
```

Root 탐색을 시작으로 하였다.

추가로 parent_offset 을 지정하였는데, 이는 디스크 io를 줄이기위해서 이렇게 사용하였다.

만약 insert 나 delete 과정에서 부모노드나 자식노드가 바뀐 경우 현재 노드의 자식 노드는 이미 메모리에 올라와 있어 바로 설정할 수 있지만, 자식노드의 부모노드를 업데이트 하는 경우 자식노드를 IO 를 하여 메모리에서 parent_offset 변수를 변경해야 한다.

이를 바로 Disk 에서 노드를 불러와 변경하지 않고, find 나 insert, delete 되는 과정에서 위치를 찾으면서 업데이트 해주면 추가 IO 를 줄일 수 있다고 생각하였다.

현재 비교가 작은 값을 비교하는 방식으로 구현되어 있어, 마지막 포인터로 이동하는 것을 구현하기 위해 Flag 변수를 통해서 마지막 노드로 갈 수 있도록 구현하였다.

```

    if (!flag)
    {
        prev_off = cur_off;
        cur_off = cur->b_f[cur->num_of_keys - 1].p_offset;
        cur = load_page(cur->b_f[cur->num_of_keys - 1].p_offset);
    }
}
cur->parent_page_offset = prev_off;
pwrite(fd, cur, sizeof(page), cur_off);

for (int i = 0; i < cur->num_of_keys; i++)
{
    if (cur->records[i].key == key)
    {
        // printf("%p\n", rt->parent_page_offset);
        // printf("entire page: %d, rt offset: %p, cur offset: %p, n", rt->parent_page_offset, rt->parent_page_offset, cur->parent_page_offset, cur->parent_page_offset);
        // printTree2(hp->rpo);
        // printTree();
        strcpy(result, cur->records[i].value);
        free(cur);
        return result;
    }
}
return 0;
}

```

마지막 leaf node 에서 key 를 찾은 경우 동적할당해둔 result 에 해당 value 를 복사하여 리턴한다. 못찾은 경우 0(NULL) 값을 리턴한다.

2. db_Insert 함수

key 를 통해서 Insert 가 필요한 위치를 찾는 로직은 위의 find 함수와 동일하다.

```

int db_insert(int64_t key, char *value)
{
    if (rt == NULL)
    {
        record *rec = create_rec(key, value);
        start_new_file(*rec);
        return 0;
    }

    page *cur = load_page(hp->rpo); // rt 로 설정 처음에
    off_t cur_off = hp->rpo; // 현재 offset
    off_t prev_off = 0;
    int cur_pos;

```

Find 와 다른 점은 추가로 **cur_pos** 변수와 **cur_off** 변수를 사용했다는 점인데, 이 변수를 사용한 이유는 부모 노드에서 현재 노드가 몇 번째에 위치한 키를 통해서 이동이 되고 있는지, 또 그 포인터의 값은 어떤 값을 가지는 지 저장하기 위해 사용하였다.

```

int db_insert(int64_t key, char *value)
{
    int pos = cur->num_of_keys;

    if (cur->is_leaf && cur->num_of_keys >= LEAF_MAX) // 현재 page 는 leaf, 꽉차있는 경우
    {
        record tmp[LEAF_MAX + 1];

        for (int i = 0; i < LEAF_MAX; i++)
        {
            if (key == cur->records[i].key)
            {
                // printf("no input\n");
                return -1;
            }
            if (key < cur->records[i].key)
            {
                pos = i;
                break;
            }
            tmp[i] = cur->records[i];
        }

        for (int i = LEAF_MAX; i > pos; i--)
        {
            tmp[i] = cur->records[i - 1];
        }

        tmp[pos].key = key;
        strcpy(tmp[pos].value, value); // 새롭게 정렬

        if (cur_off == hp->rpo) // 루트 노드가 leaf 의 경우 바로 스플릿
        {
            off_t new_off = new_page();
            page *new_p = load_page(new_off);
            off_t new_off2 = new_page();
            page *new_p2 = load_page(new_off2);

            for (int i = 0; i <= LEAF_MAX; i++) // split

```

현재 insert 할 Key 가 leaf 노드의 어느 위치에 존재해야 하는지 판단하기 위해 pos 변수를 사용한다.

현재 node 가 LEAF_MAX 만큼 key 가 존재한다면 insert 할 공간이 없는 있는 경우이다.

스플릿이나 **key rotation** 이 필요한 경우이다. 해당 작업을 수행하기 위해 tmp 임시 배열에 insert 할 키까지 모두 담아 둔다.

이후 현재 leaf 노드가 root 라면 바로 split 을 통해 나눠준다.

```

        for (int i = 0; i <= LEAF_MAX; i++) // split
        {
            if (i <= LEAF_MAX / 2)
            {
                new_p->records[i] = tmp[i];
                new_p->num_of_keys++;
            }
            else
            {
                new_p2->records[i - (LEAF_MAX / 2) - 1] = tmp[i];
                new_p2->num_of_keys++;
            }
        }

        new_p->is_leaf = 1;
        new_p2->is_leaf = 1;
        new_p->next_offset = new_off2;
        new_p2->next_offset = 0;
        new_p->parent_page_offset = hp->rpo;
        new_p2->parent_page_offset = hp->rpo;
        rt->is_leaf = 0;
        rt->num_of_keys = 1;
        rt->b_f[0].key = new_p2->records[0].key;
        rt->b_f[0].p_offset = new_off2;
        rt->next_offset = new_off;
        pwrite(fd, rt, sizeof(page), hp->rpo);
        free(rt);
        rt = load_page(hp->rpo);
        pwrite(fd, new_p, sizeof(page), new_off);
        pwrite(fd, new_p2, sizeof(page), new_off2);
        free(new_p2);
        free(new_p);
        free(cur);
    }
}
else

```

루트 노드가 LEAF 이며 바로 스플릿 하는 경우 추가 **page**가 두 개 필요하므로, 두 개의 page 를 추가로 load 하여 각각 오른쪽, 왼쪽으로 나눈 뒤 절반씩 나눠서 갖도록 한다.

루트의 처음 key는 오른쪽 자식의 0번 째 key 를 갖도록하고 next_offset 은 왼쪽 자식의 offset 을 가리키도록 설정하여 잘 연결한다.

변경 사항을 모두 disk 에 pwrite 를 통해 저장하고 rt 도 새롭게 load 하여 변경 사항을 모두 적용한다.

```

else
{ // sibling 이 존재하는 경우 == 부모 inner 노드가 존재하는 경우
    page *prev = load_page(cur->parent_page_offset);
    int flag = 0;

    // printf("GOOD cur_pos : %d, parent key: %d\n", cur_pos, prev->b_f[0])
    if (cur_pos < prev->num_of_keys) // 가장 오른쪽이 아닌 경우
    {
        flag = 1;
        page *right = load_page(prev->b_f[cur_pos].p_offset);
        if (right->num_of_keys == LEAF_MAX) // 오른쪽이 꽉 차있는 경우 왼쪽을 봐야
            flag = 0;
        else // 아닌 경우 rotation 진행.
        {
            // printf("right\n");
            for (int i = 0; i < LEAF_MAX; i++)
            {
                cur->records[i] = tmp[i];
            }

            for (int i = right->num_of_keys; i > 0; i--)
            {
                right->records[i] = right->records[i - 1];
            }

            right->records[0] = tmp[LEAF_MAX];
            right->num_of_keys++;
            if (cur_pos > 0)
            {
                prev->b_f[cur_pos - 1].key = cur->records[0].key;
                prev->b_f[cur_pos].key = right->records[0].key;
            }

            pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
            free(prev);
            pwrite(fd, right, sizeof(page), prev->b_f[cur_pos].p_offset);
            free(right);
            pwrite(fd, cur, sizeof(page), cur_off);
            free(cur);
        }
    }
}

```

현재 노드가 root가 아니라면 왼쪽, 오른쪽 leaf_node 를 확인하여 key-rotation 이 가능한지 아닌지 판단하기 위해 flag 변수를 사용한다.

현재 cur_pos 가 가장 오른쪽에 있는 경우, key-rotation이 불가능하다고 생각하고 왼쪽을 보도록 flag 를 0으로 리셋한다.

오른쪽 leaf_node가 존재하는 경우 확인하고 오른쪽 leaf node 가 모두 차있는 상태라면 key-rotation이 불가능하므로 flag 를 왼쪽을 보도록 0으로 리셋한다.

오른쪽이 비어있는 경우 오른쪽 노드의 key 를 모두 한칸씩 밀어내고 왼쪽에 삽입, 부모의 key 도 업데이트한다.

```

if (flag == 0 || cur_pos == prev->num_of_keys) // 오른쪽 sibling이 없거나 오른쪽 sibling의 자리가
{
    if (cur_pos == 0) // 왼쪽도 없는 경우(cur == 가장 왼쪽의 경우) 설정
        flag = 2;
    else
    {
        page *left;
        off_t left_off;
        if (cur_pos == 1) // cur 의 왼쪽이 첫번째(0)인 경우
        {
            left = load_page(prev->next_offset);
            left_off = prev->next_offset;
        }
        else
        {
            left = load_page(prev->b_f[cur_pos - 2].p_offset);
            left_off = prev->b_f[cur_pos - 2].p_offset;
        }
        if (left->num_of_keys == LEAF_MAX) // 왼쪽 노드의 키가 다 차있는 경우 스플릿
            flag = 2;
        else // 아닌 경우 rotation 진행.
        {
            // printf("left\n");
            for (int i = 0; i < LEAF_MAX; i++)
            {
                cur->records[i] = tmp[i + 1];
            }

            prev->b_f[cur_pos - 1].key = cur->records[0].key;
            left->records[left->num_of_keys] = tmp[0];
            left->num_of_keys++;
            pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
            free(prev);
            pwrite(fd, left, sizeof(page), left_off);
            free(left);
            pwrite(fd, cur, sizeof(page), cur_off);
            free(cur);
        }
    }
}

```

왼쪽을 봐야하는 상황이므로 현재 cur_pos 가 0(부모의 가장 왼쪽) 인 경우 스플릿을 진행할 수 있도록 flag를 2로 설정한다.

또한 왼쪽 노드도 가득 차있는 경우도 flag를 2로 설정한다

그게 아닌 경우 왼쪽노드의 마지막에 key 를 바로 삽입한다.

```

if (flag == 2) // 왼쪽 sibling이 없거나 왼쪽 sibling의 자리가 없는 경우 스플릿
{
    // printf("split\n");
    off_t new_off = new_page();
    page *new_p = load_page(new_off);
    cur->num_of_keys = 0;
    new_p->parent_page_offset = cur->parent_page_offset;
    new_p->is_leaf = 1;
    new_p->num_of_keys = 0;
    new_p->next_offset = cur->next_offset;
    cur->next_offset = new_off;
    for (int i = 0; i <= LEAF_MAX; i++) // split
    {
        if (i <= LEAF_MAX / 2)
        {
            cur->records[i] = tmp[i];
            cur->num_of_keys++;
        }
        else
        {
            // cur->records[i].key = 0;
            // strcpy(cur->records[i].value, "");
            new_p->records[i - (LEAF_MAX / 2) - 1] = tmp[i];
            new_p->num_of_keys++;
        }
    }
    prev->b_f[cur_pos - 1].key = cur->records[0].key;

    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    free(prev);
    pwrite(fd, cur, sizeof(page), cur_off);
    pwrite(fd, new_p, sizeof(page), new_off);
    split_insert(cur, new_p, new_p->records[0].key, new_off);
    free(cur);
    free(new_p);
}
}
return 0;
}

```

왼쪽 오른쪽 모두 **key rotation**을 진행할 수 없는 상황이므로 **split** 을 진행한다.

스플릿에 새로운 페이지가 필요하므로 load 한 후 현재 페이지와 새로운 페이지에 각각 key 를 나눠 담는다.

새롭게 추가 된 키가 cur 의 첫번째 위치일 수 있으므로, parent 에 업데이트 해준다.

스플릿이 필요한 상황이고 parent에 key 와 포인터를 추가 하는 경우이므로 새로운 페이지의 0번 key(추가할 key) 와 new_p 의 offset(추가할 포인터) 를 각각 split 함수에 전달한다.

```

else // 그냥 삽입 과정
{
    for (int i = 0; i < cur->num_of_keys; i++) // numofkeys 가 31 보다 작을때 수행 추가
    {
        if (cur->records[i].key == key)
        {
            // printf("no input\n");
            return -1;
        }
        if (cur->records[i].key > key)
        {
            pos = i;
            break;
        }
    }
    for (int i = cur->num_of_keys; i > pos; i--)
        cur->records[i] = cur->records[i - 1];

    cur->records[pos].key = key;
    strcpy(cur->records[pos].value, value);
    cur->num_of_keys++; // 단순 삽입

    // printf("cur offset: %p, num_of_keys: %d, key: %d, value:%s\n", cur_off, cur->num_of_keys, key, value);

    pwrite(fd, cur, sizeof(page), cur_off); // 디스크에 삽입 저장
    free(cur);
    return 0;
}

return -1;
}

```

스플릿이나 **key-rotation**이 필요하지 않은 경우는 그냥 삽입을 진행한다.

3. split_insert 함수

```
void split_insert(page *cur, page *new_p, int key, off_t new_p_off)
{
    if (cur->parent_page_offset == 0) // cur 가 root 인 경우
    {
        // printf("%d %d\n", hp->rpo, rt->parent_page_offset);
        off_t new_rt_off = new_page();
        page *new_rt = load_page(new_rt_off);

        new_rt->is_leaf = 0;
        new_rt->next_offset = new_rt_off;
        new_rt->num_of_keys = 1;
        new_rt->b_f[0].p_offset = new_p_off;
        new_rt->b_f[0].key = key;
        new_p->parent_page_offset = hp->rpo;
        cur->parent_page_offset = hp->rpo;
        new_rt->parent_page_offset = 0;
        pwrite(fd, new_rt, sizeof(page), hp->rpo);
        pwrite(fd, cur, sizeof(page), new_rt_off);
        pwrite(fd, new_p, sizeof(page), new_p_off);
        free(new_rt);
        free(rt);
        rt = load_page(hp->rpo);
        return;
    }
    page *prev = load_page(cur->parent_page_offset);
```

Root가 Non Leaf 이면서 스플릿 한 경우 새로운 페이지를 하나 더 추가한다. (총 2개 페이지 추가) 이후 **새로운 페이지를 루트** 로 설정하여 나머지를 잘 연결한다.

```
page *prev = load_page(cur->parent_page_offset);
I_R tmp[INTERNAL_MAX + 1];
int cur_pos = prev->num_of_keys;

for (int i = 0; i < prev->num_of_keys; i++)
{
    tmp[i] = prev->b_f[i];
    // printf("c[0]: %d, p[i]: %d, %d\n", cur->records[0].key, prev->b_f[i].key, i);
    if (cur->is_leaf)
    {
        if (cur->records[0].key < prev->b_f[i].key)
        {
            cur_pos = i;
            break;
        }
    }
    else
    {
        if (cur->b_f[0].key < prev->b_f[i].key)
        {
            cur_pos = i;
            break;
        }
    }
}

for (int i = prev->num_of_keys; i > cur_pos; i--)
{
    tmp[i] = prev->b_f[i - 1];
}

// printf("cur_pos: %d, key: %d\n", cur_pos, key);
tmp[cur_pos].key = key;
tmp[cur_pos].p_offset = new_p_off;
```

Root가 아닌 split의 경우 현재 스플릿한 cur 노드에 대한 위치정보가 필요하고, cur 의 **부모노드 키에 대해** 한 칸 씩 밀어내는 작업이 필요하므로 tmp 배열을 사용한다.

적절한 위치에 부모의 키를 tmp 에 모두 담아둔다.


```

if (prev->num_of_keys >= INTERNAL_MAX)
{ // parent split
    off_t new_off = new_page();
    page *new_pp = load_page(new_off);
    new_pp->is_leaf = 0;
    new_pp->num_of_keys = 0;
    new_pp->parent_page_offset = prev->parent_page_offset;
    prev->num_of_keys = 0;

    for (int i = 0; i <= INTERNAL_MAX; i++)
    {
        // printf("%d ", tmp[i]);
        if (i < INTERNAL_MAX / 2)
        {
            prev->num_of_keys++;
            prev->b_f[i] = tmp[i];
        }
        else if (i == INTERNAL_MAX / 2)
        {
            new_pp->next_offset = tmp[i].p_offset;
        }
        else
        {
            new_pp->num_of_keys++;
            new_pp->b_f[i - INTERNAL_MAX / 2 - 1] = tmp[i];
        }
    }
    // printf("tmp \n");

    {
        pwrite(fd, new_pp, sizeof(page), new_off);
        pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
        split_insert(prev, new_pp, tmp[INTERNAL_MAX / 2].key, new_off);
        free(prev);
        free(new_pp);
    }
}

```

이미 부모 노드의 key 가 모두 차있
어 자리가 없는 경우, 부모 노드 역
시 스플릿이 필요한 상황이므로, 새
로운 페이지를 Load 하여 부모 노드
의 키를 똑같이 나눈다.

재귀적으로 split 함수를 호출하여
부모가 스플릿이 되지 않을 때 까지
스플릿 과정을 진행한다.

```

else // 그냥 삽입
{
    // printf("just insert\n");
    prev->num_of_keys++;
    for (int i = 0; i < prev->num_of_keys; i++)
    {
        prev->b_f[i] = tmp[i];
        // printf("%d ", tmp[i]);
    }
    // printf("\n");
    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    free(prev);
}
}

```

부모 노드의 자리가 있는 경우 적
절한 위치에 key 와 포인터를 삽입
하여 작업을 종료한다.

4. update_key 함수

```
void update_key(int64_t key, int64_t upadate_key)
// delete 작업에서 inner key 에 해당하는 key 를 update 한다.
{
    page *cur = load_page(hp->rpo); // rt 로 설정 처음에
    off_t cur_off = hp->rpo;         // 현재 offset
    off_t prev_off = 0;
    int cur_pos;

    while (!cur->is_leaf)
    {
        cur_pos = 0;
        if (cur->parent_page_offset != prev_off)
        {
            cur->parent_page_offset = prev_off;
            pwrite(fd, cur, sizeof(page), cur_off);
        }

        if (key == cur->b_f[0].key)
        {
            cur->b_f[0].key = upadate_key;
            pwrite(fd, cur, sizeof(page), cur_off);
            free(cur);
            return;
        }
        if (key < cur->b_f[0].key)
        {
            prev_off = cur_off;
            cur_off = cur->next_offset;
            cur = load_page(cur->next_offset);
        }
    }
}
```

Update_key 함수는 leaf 노드의 가장 앞부분의 key 가 Delete 되었을 때 Tree 에서 해당 key 에 대한 nonleaf key 를 업데이트 해주는 함수이다.

Find 로직은 db_find 함수와 동일하고, 동일한 키가 존재할 때 update 를 해주고 함수는 종료된다.

5. db_delete 함수

```
int db_delete(int64_t key)
{
    if (rt == NULL)
        return -1;

    page *cur = load_page(hp->rpo); // rt 로 설정
    off_t cur_off = hp->rpo;         // 현재 offset
    off_t prev_off = 0;
    int cur_pos;

    if (pos == cur->num_of_keys) // not exist
    {
        free(cur);
        // printf("Not exist\n");
        return -1;
    }
}
```

첫 delete 할 key 를 찾는 과정은 insert 함수와 모두 동일하다.

Key 가 존재하지 않는 경우, -1 을 리턴하도록 설정.

```

cur->num_of_keys--;
for (int i = pos; i < cur->num_of_keys; i++)
{
    cur->records[i] = cur->records[i + 1];
}

if (pos == 0) // 0번 키를 삭제한 경우 key 업데이트
{
    update_key(key, cur->records[0].key);
}
pwrite(fd, cur, sizeof(page), cur_off);

if (cur->num_of_keys < LEAF_MAX / 2)
{
    delete_balance(cur);
}
free(cur);
return 0;

```

Delete 하는 과정을 진행한다.

Num_of_key 를 하나 줄이고, delete 해당 키의 뒤 위치 key를 모두 한칸씩 당겨온다.

0번 key 를 삭제한 경우 위의 update 함수를 진행.

삭제한 현재 노드가 최대 key 개수의 절반보다 작다면 **delete_balance** 함수를 호출하여 **Merge, distribution**을 진행한다.

6. delete_balance 함수

```

void delete_balance(page *cur)
{
    // 재분배 가능 여부 1. 왼쪽 2. 오른쪽 확인
    // 재분배 불가능의 경우 -> merge 시도, cur 가 가장 왼쪽의 경우 오른쪽
    // 아닌경우 모두 왼쪽과 머지
    // printf("p: %d, cur: %d\n", cur->parent_page_offset, cur->num_of_keys);
    if (cur->parent_page_offset == 0 && cur->num_of_keys > 0)
        return;

    if (cur->parent_page_offset == 0 && cur->num_of_keys == 0)
    {
        page *new_rt = load_page(cur->next_offset);
        free(rt);
        // 현재 페이지가 root 의 경우, 남아있는
        // pos 에 따라서 왼쪽, 오른쪽을 선택

        new_rt->parent_page_offset = 0;
        pwrite(fd, new_rt, sizeof(page), hp->rpo);
        free(new_rt);
        rt = load_page(hp->rpo);

        return;
    }
    else

```

Cur 노드의 num_of_keys 의 수가 최대 절반보다 적어져 현재 균형을 맞춰야 하는 상태이다.

루트 노드에 대해서 num_of_keys 가 부족한 경우, key 개수 제한이 없기 때문에, 0인 경우를 제외하고 처리를 하지 않는다.

루트의 key 개수가 0이 된 경우, 새로운 자식 노드가 root가 되어야 하는 상황이다. Merge 는 항상 결과가 왼쪽 node 에 저장되기 때문에, **루트의 1번 자식 노드**를 새로운 **root** 로 지정한다.

```

else
{
    int flag = 0;
    page *prev = load_page(cur->parent_page_offset);

    int cur_pos = prev->num_of_keys;

    for (int i = 0; i < prev->num_of_keys; i++)
    {
        // printf("c[0]: %d, p[i]: %d, %d\n", cur->records[0].key, prev->b_f[i].key, i);
        if (cur->is_leaf)
        {
            if (cur->records[0].key < prev->b_f[i].key)
            {
                cur_pos = i;
                break;
            }
        }
        else
        {
            if (cur->b_f[0].key < prev->b_f[i].key)
            {
                cur_pos = i;
                break;
            }
        }
    }
}

```

Root 노드가 아닌 경우 현재 노드가 부모에서 몇 번째 노드인지 확인이 필요하다.

```

if (cur_pos != 0) // 현재 가장 왼쪽이 아닌 경우, 왼쪽을 확인해야함.
{
    flag = 1;
    page *left;
    if (cur_pos == 1)
        left = load_page(prev->next_offset);
    else
        left = load_page(prev->b_f[cur_pos - 2].p_offset);

    if (left->is_leaf && left->num_of_keys <= LEAF_MAX / 2) // left 에서 받아올 수 없음
        flag = 0;
    else if (!left->is_leaf && left->num_of_keys <= INTERNAL_MAX / 2)
        flag = 0;
    else
    { // left 에서 key 하나 받아오기 진행 left 는 항상 존재.
        // printf("left distribute\n");
        if (left->is_leaf) // leaf node 의 경우
        {
            for (int i = cur->num_of_keys; i > 0; i--)
                cur->records[i] = cur->records[i - 1];

            cur->records[0] = left->records[left->num_of_keys - 1];
            left->num_of_keys--;
            cur->num_of_keys++;
            prev->b_f[cur_pos - 1].key = cur->records[0].key;
            pwrite(fd, left, sizeof(page), prev->b_f[cur_pos - 2].p_offset);
            pwrite(fd, cur, sizeof(page), prev->b_f[cur_pos - 1].p_offset);
            pwrite(fd, prev, sizeof(page), cur->parent_page_offset); // good to have
            free(left);
            free(prev);
        }
    }
}

```

먼저 왼쪽 sibling을 봐야 하므로, 현재 위치가 0이 아니면 왼쪽이 존재한다. 왼쪽 노드의 num_of_keys 가 절반보다 작지 않은 경우, left의 key 하나를 받아와서 트리 균형을 유지한다.

절반보다 작은 경우, 왼쪽에서 받아오지 못하므로 오른쪽 노드를 봐야하기 때문에 flag 0 으로 설정.

현재 노드가 leaf, nonleaf 두가지 일 수 있으므로 경우를 나눈다

```

else
{
    // printf("test\n");
    for (int i = cur->num_of_keys; i > 0; i--)
        cur->b_f[i] = cur->b_f[i - 1];
    cur->b_f[0].p_offset = cur->next_offset;
    cur->b_f[0].key = prev->b_f[cur_pos - 1].key;
    prev->b_f[cur_pos - 1].key = left->b_f[left->num_of_keys - 1].key;
    cur->next_offset = left->b_f[left->num_of_keys - 1].p_offset;
    prev->b_f[cur_pos - 1].key = left->b_f[left->num_of_keys - 1].key;
    cur->num_of_keys++;
    left->num_of_keys--;
    pwrite(fd, left, sizeof(page), prev->b_f[cur_pos - 2].p_offset);
    pwrite(fd, cur, sizeof(page), prev->b_f[cur_pos - 1].p_offset);
    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    free(left);
    free(prev);
}

```

Leaf 의 경우 key 하나를 받아오는 작업만 한다.

Nonleaf 의 경우 포인터를 받아오면서 key 값도 함께 업데이트를 해주어야 한다. Left 의 key 를 부모로, 부모의 key 를 현재 노드로 rotation 하듯이 받아온다.

```

if (flag == 0) // 오른쪽에서 key 하나 받아오기 진행
{
    flag = 1;
    if (cur_pos >= prev->num_of_keys) // 가장 오른쪽에 있는 경우 오른쪽이 없음. 머지 해야하는 상황
        flag = 2;
    else
    {
        page *right = load_page(prev->b_f[cur_pos].p_offset);

        if (right->is_leaf && right->num_of_keys <= LEAF_MAX / 2) // 오른쪽 노드가 두
            flag = 2;
        else if (!right->is_leaf && right->num_of_keys <= INTERNAL_MAX / 2)
            flag = 2;
        else
        { // 오른쪽에서 가져오기 진행
            // printf("right distribute\n");
            if (right->is_leaf) // leaf node 의 경우
            {
                cur->records[cur->num_of_keys] = right->records[0];
                for (int i = 0; i < right->num_of_keys; i++)
                    right->records[i] = right->records[i + 1];

                right->num_of_keys--;
                cur->num_of_keys++;
                prev->b_f[cur_pos].key = right->records[0].key;
                pwrite(fd, right, sizeof(page), prev->b_f[cur_pos].p_offset);
                pwrite(fd, cur, sizeof(page), prev->b_f[cur_pos - 1].p_offset);
                pwrite(fd, prev, sizeof(page), cur->parent_page_offset); // good
                free(right);
                free(prev);
            }
        }
    }
}

```

오른쪽에서 포인터를 받아오는 경우이다.

오른쪽 역시 오른쪽의 num_of_keys 값이 반절보다 적으면 진행하지 못하므로 merge 하기 위해서 flag 2로 설정한다.

왼쪽과 동일하게 leaf nonleaf 구분하여 받아오고 leaf의 경우 키 값만 받아오고, 오른쪽 노드의 키를 한칸씩 앞으로 당겨온다.

당겨왔으므로, 0번 key 를 위로 올려준다.

nonleaf 의 경우 왼쪽과 동일하게 key 를 rotation 하듯이 가져온다.

```

else // 잘 됨
{
    cur->b_f[cur->num_of_keys].p_offset = right->next_offset;
    cur->b_f[cur->num_of_keys].key = prev->b_f[cur_pos].key;
    prev->b_f[cur_pos].key = right->b_f[0].key;

    right->next_offset = right->b_f[0].p_offset;
    for (int i = 0; i < right->num_of_keys; i++)
        right->b_f[i] = right->b_f[i + 1];

    cur->num_of_keys++;
    right->num_of_keys--;
    pwrite(fd, right, sizeof(page), prev->b_f[cur_pos].p_offset);
    pwrite(fd, cur, sizeof(page), prev->b_f[cur_pos - 1].p_offset);
    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    free(right);
    free(prev);
}

```

```

if (flag == 2) // merge 해야하는 상황.
{
    if (cur_pos == 0) // 현재 노드가 가장 왼쪽이라면 오른쪽과 merge 해야함.
    {
        page *right = load_page(prev->b_f[cur_pos].p_offset);
        // printf("right merge\n");
        if (!cur->is_leaf) // INTERNAL node 에 대해서 merge
        {
            cur->b_f[cur->num_of_keys].key = prev->b_f[cur_pos].key;
            cur->b_f[cur->num_of_keys].p_offset = right->next_offset;

            for (int i = cur->num_of_keys + 1; i < cur->num_of_keys + right->num_of_keys; i++)
            {
                cur->b_f[i] = right->b_f[i - cur->num_of_keys - 1];
            }
            cur->num_of_keys += right->num_of_keys + 1;

            usetofree(prev->b_f[0].p_offset);
            prev->num_of_keys--;
            for (int i = 0; i < prev->num_of_keys; i++)
                prev->b_f[i] = prev->b_f[i + 1];

            pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
            pwrite(fd, cur, sizeof(page), prev->next_offset);

            free(right);

            if (prev->num_of_keys < INTERNAL_MAX / 2)
            {
                delete_balance(prev);
            }
        }
    }
}

```

Merge 가 필요한 상황이
다. 우선적으로 왼쪽 sibling
에 현재 노드를 merge 한
다.

예외가 있다면 현재 노드
가 가장 왼쪽노드의 경우
right sibling 과 머지한다.

오른쪽과 머지 하지만, 오
른쪽의 노드를 현재 노드로
가져온다고 보면 된다. 그렇
게 되면 항상 결과가 왼쪽
이 남게 된다.

이후 오른쪽 노드를 free
하여 헤더에 빈 page로 관
리한다.

```

else // leaf 노드에 대해 merge
{
    for (int i = cur->num_of_keys; i < cur->num_of_keys + right->num_of_keys; i++)
        cur->records[i] = right->records[i - cur->num_of_keys];

    cur->num_of_keys += right->num_of_keys;
    cur->next_offset = right->next_offset;

    usetofree(prev->b_f[0].p_offset);
    prev->num_of_keys--;
    for (int i = 0; i < prev->num_of_keys; i++)
        prev->b_f[i] = prev->b_f[i + 1];

    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    pwrite(fd, cur, sizeof(page), prev->next_offset);
    free(right);

    if (prev->num_of_keys < INTERNAL_MAX / 2)
    {
        delete_balance(prev);
    }
}

free(prev);

```

Leaf 의 경우 오른쪽 노
드의 key 와 record 를
모두 이어붙인다. Non
leaf leaf 모두 prev의
num_of keys 를 줄이고,
절반 조건이 맞는지 확인
한다. 맞지 않는다면 prev
에 대해서 balance 맞추
기를 재귀적으로 한다.

```

else
// 범위를 축소하려면 클릭합니다. 왼쪽과 merge
// printf("left merge\n");
page *left;
off_t left_off;
if (cur_pos == 1)
{
    left_off = prev->next_offset;
    left = load_page(left_off);
}
else
{
    left_off = prev->b_f[cur_pos - 2].p_offset;
    left = load_page(left_off);
}
if (!cur->is_leaf)
{
    left->b_f[left->num_of_keys].key = prev->b_f[cur_pos - 1].key;
    left->b_f[left->num_of_keys].p_offset = cur->next_offset;
    left->num_of_keys++;
    prev->num_of_keys--;

    for (int i = 0; i < cur->num_of_keys; i++)
    {
        left->b_f[left->num_of_keys] = cur->b_f[i];
        left->num_of_keys++;
    }
    usetofree(prev->b_f[cur_pos - 1].p_offset);
    for (int i = cur_pos - 1; i < prev->num_of_keys; i++)
    {
        prev->b_f[i] = prev->b_f[i + 1]; //
    }
    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    pwrite(fd, left, sizeof(page), left_off);
    free(left);
    if (prev->num_of_keys < INTERNAL_MAX / 2)
    {
        delete_balance(prev);
    }
}
}

```

왼쪽과 머지한다면, nonleaf
의 경우 현재 key 와 포인터를
left 에 이어 붙인뒤, prev 의
노드구성을 하나씩 다 당겨온
다. 또 페이지를 하나 제거한
다.

동일하게 prev 에 대해 검사
진행, 재귀적으로 해결한다.

```

else
{ // leaf node 의 경우

    for (int i = 0; i < cur->num_of_keys; i++)
    {
        left->records[left->num_of_keys] = cur->records[i];
        left->num_of_keys++;
    }
    left->next_offset = cur->next_offset;
    prev->num_of_keys--;
    usetofree(prev->b_f[cur_pos - 1].p_offset);
    for (int i = cur_pos - 1; i < prev->num_of_keys; i++)
    {
        prev->b_f[i] = prev->b_f[i + 1]; //
    }
    pwrite(fd, prev, sizeof(page), cur->parent_page_offset);
    pwrite(fd, left, sizeof(page), left_off);
    free(left);
    if (prev->num_of_keys < INTERNAL_MAX / 2)
    {
        delete_balance(prev);
    }
}
free(prev);
}
}
}
}

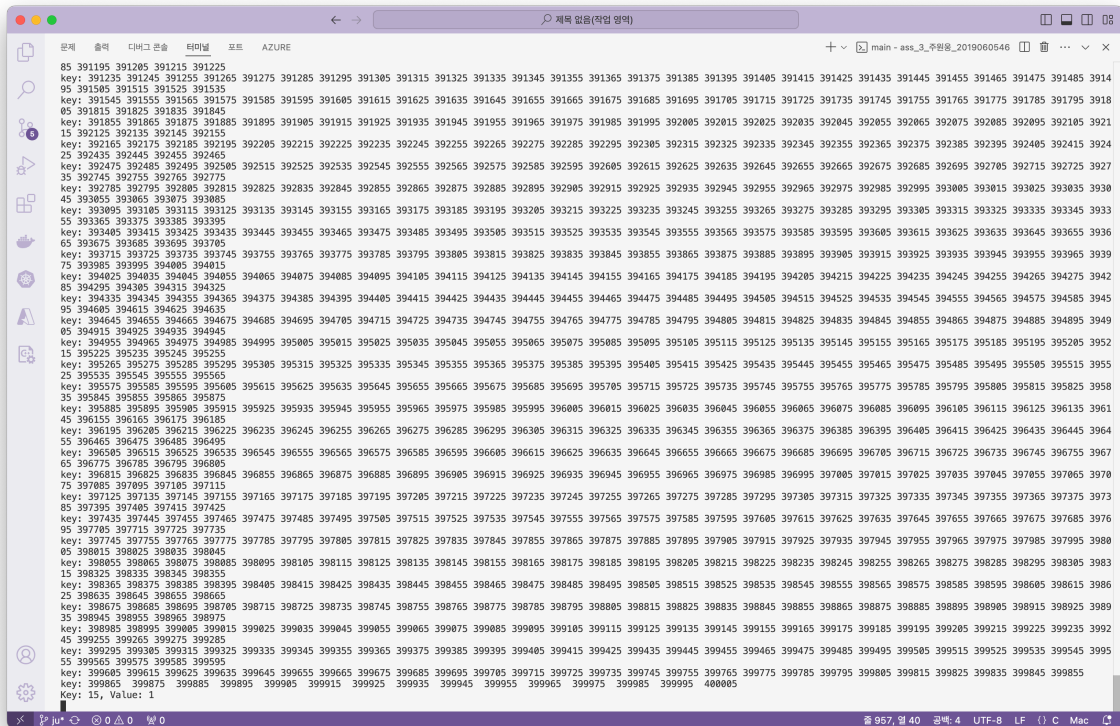
```

Leaf 의 경우도 동일하게 처리
한다.

• Result

```
f15
p: 0 c: 4096 n: 1 i key: 3735
p: 4096 c: 110592 n: 3 i key: 475 940 2800
p: 110592 c: 36864 n: 2 i key: 165 320
p: 36864 c: 8192 n: 31 r key: 10 15 20 25 30 35 40 45 50 55 60 65 70 75 80 85 90 95 100 105 110 115 120 125 130 135 140 145 150 155 160
p: 36864 c: 98304 n: 31 r key: 165 170 175 180 185 190 195 200 205 210 215 220 225 230 235 240 245 250 255 260 265 270 275 280 285 290 295 300 305 310 315
p: 36864 c: 12288 n: 31 r key: 320 325 330 335 340 345 350 355 360 365 370 375 380 385 390 395 400 405 410 415 420 425 430 435 440 445 450 455 460 465 470
p: 4096 c: 102400 n: 2 i key: 630 785
p: 36864 c: 94208 n: 31 r key: 475 480 485 490 495 500 505 510 515 520 525 530 535 540 545 550 555 560 565 570 575 580 585 590 595 600 605 610 615 620 625
p: 36864 c: 16384 n: 31 r key: 630 635 640 645 650 655 660 665 670 675 680 685 690 695 700 705 710 715 720 725 730 735 740 745 750 755 760 765 770 775 780
p: 36864 c: 98112 n: 31 r key: 785 790 795 800 805 810 815 820 825 830 835 840 845 850 855 860 865 870 875 880 885 890 895 900 905 910 915 920 925 930 935
p: 110592 c: 32768 n: 3 i key: 1095 1175 2620
p: 32768 c: 20480 n: 31 r key: 940 945 950 955 960 965 970 975 980 985 990 995 1000 1005 1010 1015 1020 1025 1030 1035 1040 1045 1050 1055 1060 1065 1070 1075 1080 1085 1090
p: 32768 c: 81920 n: 16 r key: 1095 1100 1105 1110 1115 1120 1125 1130 1135 1140 1145 1150 1155 1160 1165 1170
p: 32768 c: 24576 n: 28 r key: 1175 1180 1185 1190 1195 1200 1205 1210 1215 1220 1225 1230 1235 1240 1245 2490 2500 2510 2520 2530 2540 2550 2560 2570 2580 2590 2600 2610
p: 32768 c: 49152 n: 15 r key: 2620 2660 2670 2680 2690 2700 2710 2720 2730 2740 2750 2760 2770 2780 2790
p: 110592 c: 69632 n: 2 i key: 3110 3435
p: 53248 c: 57344 n: 31 r key: 2800 2810 2820 2830 2840 2850 2860 2870 2880 2890 2900 2910 2920 2930 2940 2950 2960 2970 2980 2990 3000 3010 3020 3030 3040 3050 3060 3070 3080 3090 3100
p: 96932 c: 61440 n: 29 r key: 3110 3120 3130 3140 3150 3160 3170 3180 3190 3200 3210 3220 3230 3240 3250 3260 3290 3300 3310 3320 3330 3340 3350 3360 3370 3380 3390 3400 3410
p: 96932 c: 65536 n: 30 r key: 3435 3445 3455 3465 3475 3485 3495 3505 3515 3525 3535 3545 3555 3565 3575 3585 3595 3605 3615 3625 3635 3645 3655 3665 3675 3685 3695 3705 3715 3725
p: 4096 c: 163840 n: 2 i key: 4665 5595
p: 106496 c: 126976 n: 2 i key: 4045 4355
p: 96932 c: 114688 n: 31 r key: 3735 3745 3755 3765 3775 3785 3795 3805 3815 3825 3835 3845 3855 3865 3875 3885 3895 3905 3915 3925 3935 3945 3955 3965 3975 3985 3995 4005 4015 4025 4035
p: 96932 c: 118784 n: 31 r key: 4045 4055 4065 4075 4085 4095 4105 4115 4125 4135 4145 4155 4165 4175 4185 4195 4205 4215 4225 4235 4245 4255 4265 4275 4285 4295 4305 4315 4325 4335 4345
p: 126976 c: 122880 n: 31 r key: 4355 4365 4375 4385 4395 4405 4415 4425 4435 4445 4455 4465 4475 4485 4495 4505 4515 4525 4535 4545 4555 4565 4575 4585 4595 4605 4615 4625 4635 4645 4655
p: 106496 c: 143360 n: 2 i key: 4975 5285
p: 126976 c: 131072 n: 31 r key: 4665 4675 4685 4695 4705 4715 4725 4735 4745 4755 4765 4775 4785 4795 4805 4815 4825 4835 4845 4855 4865 4875 4885 4895 4905 4915 4925 4935 4945 4955 4965
p: 126976 c: 135168 n: 31 r key: 4975 4985 4995 5005 5015 5025 5035 5045 5055 5065 5075 5085 5095 5105 5115 5125 5135 5145 5155 5165 5175 5185 5195 5205 5215 5225 5235 5245 5255 5265 5275
p: 143360 c: 139264 n: 31 r key: 5285 5295 5305 5315 5325 5335 5345 5355 5365 5375 5385 5395 5405 5415 5425 5435 5445 5455 5465 5475 5485 5495 5505 5515 5525 5535 5545 5555 5565 5575 5585
p: 163840 c: 159744 n: 2 i key: 5905 6065
p: 143360 c: 147456 n: 31 r key: 5595 5605 5615 5625 5635 5645 5655 5665 5675 5685 5695 5705 5715 5725 5735 5745 5755 5765 5775 5785 5795 5805 5815 5825 5835 5845 5855 5865 5875 5885 5895
p: 143360 c: 155136 n: 16 r key: 5905 5915 5925 5935 5945 5955 5965 5975 5985 5995 6005 6015 6025 6035 6045 6055
p: 159744 c: 155648 n: 28 r key: 6065 6075 6085 6095 6105 6115 6125 6135 6145 6155 6165 6175 6185 6195 6205 6215 6225 6235 6245 6255 6265 6275 6285 6295 6305 6315 6325 6335
Key: 15, Value: 1
```

직접 printTree 함수를 만들어 테스트케이스를 만들고, Internal_MAX 값을 수정하여 테스트한 결과 잘 작동하는 것을 보았다.



7만 input 에도 잘 작동하는 것을 보았다

• Trouble shooting

1. 연결과정이 매우 케이스가 많아서 복잡했다.
2. 또한 internal_max 의 값이 너무 커서 어떻게 해야할까 고민했는데 줄여서 구현을 하였고, 248로 다시 테스트 했을 잘 돌아가는 것을 확인하였다.

• Key-rotation insert 고찰

전반적으로 데이터가 랜덤한 key 값으로 insert 되는 경우는 key-rotation이 좋은 성능을 보일 것 같다. 그 이유는 랜덤한 키 값이므로 key-rotation으로 인한 추가적인 Disk IO 가 많이 발생하지 않을 것이기 때문이다. 스플릿을 진행하면 랜덤하게 key 값이 들어오므로 전반적으로 key-rotation으로 인한 IO 가 자주 일어나지 않을 것 같다.

하지만 순서대로 key 값이 insert 되는 경우나 고르지 않게 key 값이 insert 되는 경우는 성능 저하가 일어날 것 같다. 순서대로 key 가 insert 될 시, 대부분의 insert 작업 때마다 추가적인 disk IO 가 발생할 것이기 때문이다. 하지만 트리의 공간적 효율성을 좋을 것이다. 모든 트리의 노드가 대부분 빈공간이 없을 것이기 때문이다. 이러한 이유로 특정 상황에서는 좋지만, 어떤 상황에서는 좋지 않을 것 같다는 생각이다. 또한 성능 뿐 아니라 공간적 효율성도 따져야 한다면 key rotation이 좋을 것 같다고 생각한다.